Step by step Deploying Docker Container to Azure using Azure CLI

Prateek Singh

Wed, Sep 2 2020 (https://4sysops.com/archives/step-by-step-deploying-docker-container-to-azure-using-azure-cli/)

azure, cloud computing, docker 0

In this article we will discuss step by step creation and deployment of a docker containers in the Azure Resource Group using Azure CLI. Later we will also cover how we can start, stop, restart, or delete the container group that we created, all this is can be alternatively performed using the Graphical User Interface (GUI) of the Azure portal, but this piece will only focus on using Azure CLI.

Author Recent Posts

(https://4sysops.-com/members/prateeks-ingh/)

Prateek Singh (Https://4sysops.Com/Members/Prateeksingh/)

Prateek Singh is an infrastructure developer, an avid PowerShell blogger, and a community contributor. In 2017, FeedSpot recognized his blog RidiCurious.com (https://ridicurious.com/) as among the "Top 50 PowerShell blogs in the world." All of his PowerShell projects and modules are open-sourced at GitHub (https://github.com/prateekkumarsingh), and you can reach out to him on Twitter at @SinghPrateik (https://twitter.com/SinghPrateik).

Contents

1. What is Docker and Containers
2. What is Azure CLI
3. Creating a Resource Group
4. Creating a container/container instance
5. Stopping and Starting containers in a container group
6. Conclusion

What is Docker and Containers ^

It is prerequisite that readers have some basic understanding of Docker and containers in general, but for sake of doing it, following are simple and short definitions of each:

- **Docker:** Docker is a platform or a tool which can be used to create and deploy the applications using containers.
- **Containers**: A Container is an encapsulated unit of an application and all its dependencies such that they are independent of underlying systems, or in simpler words platform independent.

What is Azure CLI ^

Azure CLI or is a command line interface available in Azure Cloud Shell in the Azure portal that can be used to create, delete and manage the Azure resources. One of the biggest advantages of using the Azure CLI to deploy Docker containers is that you don't need to install docker in cloud shell as it comes pre-installed.

Let us start deploying a docker container to Azure using Azure CLI

Creating a Resource Group ^

We will be deploying an Azure Container instance and hence we would require a resource group in which can hold these instances. To create a resource group, please run following command in PowerShell on Azure Cloud Shell:

```
az group create --name RG-Container --location CentralIndia
```

and the result will look like below attached screenshot

```
PS /> az group create --name RG-Container --location CentralIndia
{
  "id": "/subscriptions/a233a642-f4b5-4687-9b4c-9e12e2c3c80d/resourceGroups/RG-Container",
  "location": "centralindia",
  "managedBy": null,
  "name": "RG-Container",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

(https://4sysops.com/wp-content/uploads/2020/09/Create-resource-group-in-Azure.png)

Create resource group in Azure

To view the resource group, you can also run the following command with **show** keyword.

```
az group show --name RG-Container -o table
```

```
PS /> az group show --name RG-Container -o table
Location      Name
------------  ------------
centralindia  RG-Container
```

(https://4sysops.com/wp-content/uploads/2020/09/View-resource-group.png)

View resource group

Creating a container/container instance ^

Once a resource group is created where we can deploy an azure container instance, lets proceed with the next step of creating a container. We will be using following attributes while creating azure container instance:

- **--resource-group** : to specify the resource group in which the azure container instance will be deployed
- **--name** : to specify the name of the azure container instance
- **--image** : to specify the docker container image
- **--dns-name-label** : to specify the DNS name label so as the web app can be accessed publicly over the internet and this has to be unique within the region of azure where we are creating the container instance.
- **--ports** : to specify the port on which the web app will be available to communicate.

In this example we will be using public image from Docker hub: nginx (https://hub.docker.com/_/nginx) , which is a free open-source web server.

For the convenience let's put the name of resource group and the container in the variables and run the following in Azure cloud shell.

```
$resourcegroup = 'RG-Container'
$ContainerName = 'democontainer’
```

Once that is done, we will run the following command to create a container instance using Azure CLI:

```
az container create `
    --resource-group $resourcegroup `
    --name $containerName `
    --image nginx `
    --dns-name-label $containerName `
    --port 80
```

After some time once the command is executed successfully, you will see the results as below:

```
PowerShell
PS /> $resourcegroup = 'RG-Container'
PS /> $ContainerName = 'democontainer'
PS />
PS /> az container create `
>>     --resource-group $resourcegroup `
>>     --name $containerName `
>>     --image nginx `
>>     --dns-name-label $containerName `
>>     --port 80
{- Finished ..
  "containers": [
    {
      "command": null,
      "environmentVariables": [],
      "image": "nginx",
      "instanceView": {
        "currentState": {
          "detailStatus": "",
          "exitCode": null,
          "finishTime": null,
          "startTime": "2020-08-23T21:27:05+00:00",
          "state": "Running"
        },
        "events": [
          {
```

(https://4sysops.com/wp-content/uploads/2020/09/Create-container-using-Azure-CLI.png)

Create container using Azure CLI

And there would be lot more information mentioned in it. The first thing we will need to check in the results is provisioning state. If it shows as succeeded it means the container is successfully created and deployed in the mentioned resource group. The provisioning status result will look as demonstrated in the following image:

```
    "fqdn": "democontainer.centralindia.azurecontainer.io",
    "ip": "20.40.41.251",
    "ports": [
      {
        "port": 80,
        "protocol": "TCP"
      }
    ],
    "type": "Public"
  },
  "location": "centralindia",
  "name": "democontainer",
  "networkProfile": null,
  "osType": "Linux",
  "provisioningState": "Succeeded",
  "resourceGroup": "RG-Container",
  "restartPolicy": "Always",
  "tags": {},
  "type": "Microsoft.ContainerInstance/containerGroups",
  "volumes": null
}
PS />
```

(https://4sysops.com/wp-content/uploads/2020/09/Verify-container-provisioning-was-successful.png)

Verify container provisioning was successful

We can also check the provisioning state, IP and FQDN of the container using following command:

```
1.  az container show `
2.          --resource-group $resourcegroup `
3.          --name $ContainerName `
4.          --query "{FQDN:ipAddress.fqdn,IPAddress:ipAddress.ip,Provi-
sioningState:provisioningState}" `
5.          --out table
```

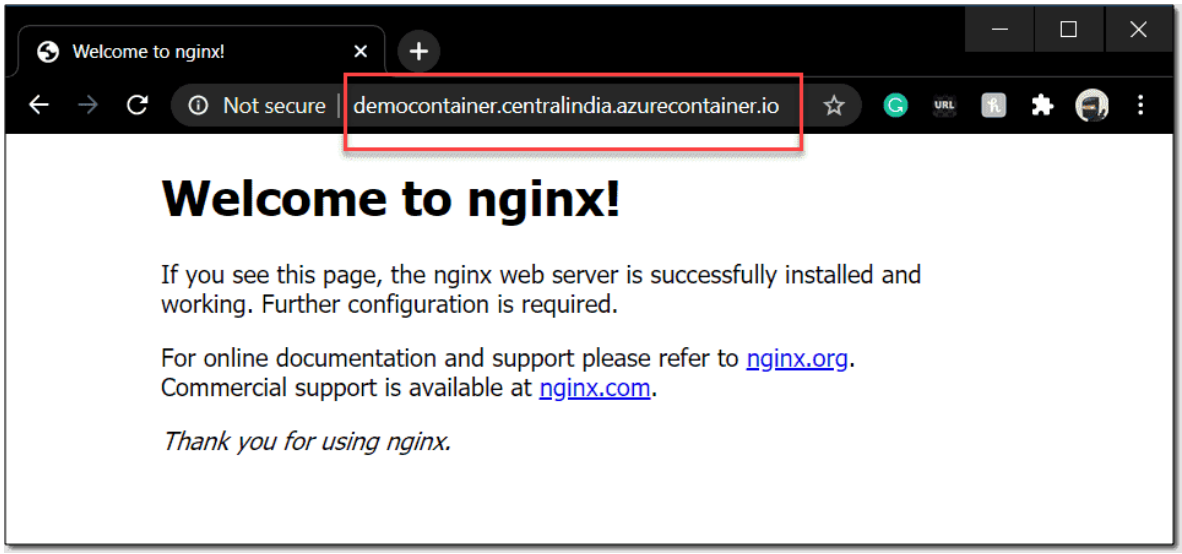
```
PowerShell
PS /> az container show `
>>        --resource-group $resourcegroup `
>>        --name $ContainerName `
>>        --query "{FQDN:ipAddress.fqdn,IPAddress:ipAddress.ip,ProvisioningState:provisioningState}" `
>>        --out table

FQDN                                        IPAddress     ProvisioningState
------------------------------------------  ------------  -------------------
democontainer.centralindia.azurecontainer.io  20.40.41.251  Succeeded
PS />
```

(https://4sysops.com/wp-content/uploads/2020/09/Show-containers-deployed-using-Azure-CLI.png)

Show containers deployed using Azure CLI

Now let us quickly verify if the web server was successfully deployed in a docker container to Azure by coping the FQDN retrieved, as demonstrated in the above example, by simply using the URL in any browser of your choice. In our example the FQDN is 'democontainer.centralindia.azurecontainer.io' and once we open it, the resulting page should look like in the following example:



(https://4sysops.com/wp-content/uploads/2020/09/Verify-the-web-server-using-the-FQDN-of-of-the-Azure-container.png)

Verify the web server using the FQDN of of the Azure container

Furthermore, the same can be tried using the public IP address retrieved from the above results of "az container show". If you get the similar page like above means your web app and container instance is successfully deployed.

Now as the container is deployed, let's look at some examples on how we can manage them using Azure CLI

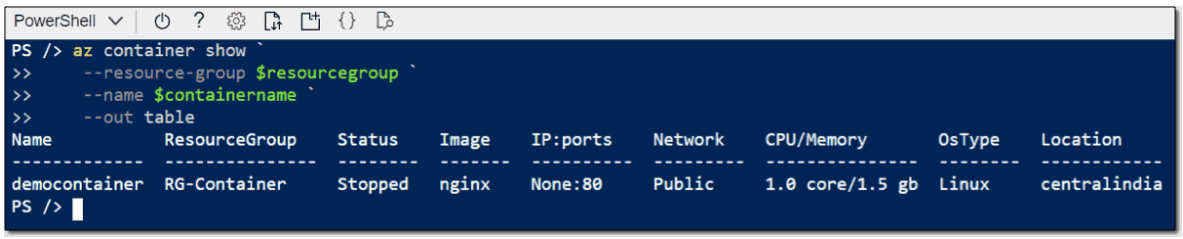
Stopping and Starting containers in a container group ^

To stop the containers in a resource group, we simply use 'container stop' as demonstrated in the following example:

```
az container stop `
    --resource-group $resourcegroup `
    --name $containername
```

and to verify the results, we can use following command and check the status which would be showing stopped:

```
az container show `
    --resource-group $resourcegroup `
    --name $containername `
    --out table
```



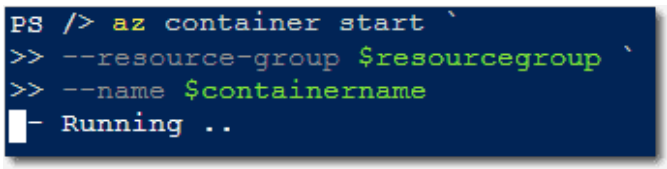
(https://4sysops.com/wp-content/uploads/2020/09/Checking-the-stopped-Azure-Container.png)

Checking the stopped Azure Container

To start it again, we can use following command with 'start' keyword:

```
az container start `
    --resource-group $resourcegroup `
    --name $containername
```

While it would be processing the request of start you will see following message on your screen:



(https://4sysops.com/wp-content/uploads/2020/09/Starting-a-container-using-Azure-CLI.png)

Starting a container using Azure CLI

Once started you can run the "az container show" command again with the output in table format which will show you the status as running. While we have checked how we can stop and start all the containers in a container group let's see how we can restart it directly with one single command instead of stopping and starting:

```
az container restart `
    --resource-group $resourcegroup `
    --name $containername
```

Again, while it would be processing the request CLI will show you the message as shown in below image:

```
PS / > az container restart `
>> --resource-group $resourcegroup `
>> --name $containername `
>>
- Running ..
```

(https://4sysops.com/wp-content/uploads/2020/09/Restarting-a-container-using-Azure-CLI.png)

Restarting a container using Azure CLI

Once you are done with the work and simply want to get rid of container instances you have deployed, then you can simply delete a container using the following command:

```
az container delete `
    --resource-group $resourcegroup `
    --name $containername
```

As soon as we will hit enter it will prompt us to review our action and agree or cancel the command

```
PS / > az container delete `
>> --resource-group $resourcegroup `
>> --name $containername
Are you sure you want to perform this operation? (y/n):
```

(https://4sysops.com/wp-content/uploads/2020/09/Deleting-a-container-using-Azure-CLI.png)

Deleting a container using Azure CLI

Once the deletion is completed, you can review if the concerned container group is existing or not through the same *az container show* command.

Subscribe to 4sysops newsletter!

Email address

Subscribe

Conclusion ^

Pulling an container image from Docker hub and then deploying a container using the same image in Azure CLI is not only simple as just running few Azure CLI commands but, it is at the same time effective since you get Docker preinstalled in Azure

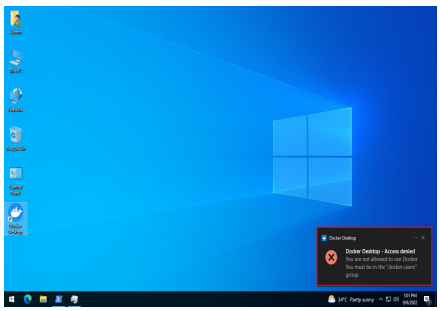
cloud shell where we are using the command line interface. More importantly you get all the goodness of Azure cloud natively like Azure portal-based management, configuration and inbuild monitoring.

0

Want to write for 4sysops? We are looking for new authors. (https://4sysops.-com/write-for-4sysops//?utm_source=4sysops&utm_medium=endpage&utm_campaign=Author)

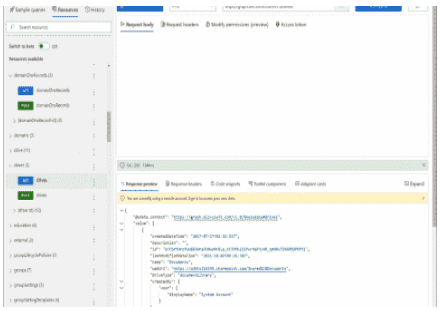
4sysops members can earn and read without ads! (https://4sysops.com/join/?utm_source=4sysops&utm_medium=endpage&utm_campaign=NoAds)

RELATED ARTICLES



(https://4sysops.com/archive docker-in-windows-10-and-windows-11/)

Install Docker on Windows 10 and Windows 11



(https://4sysops.com/archive graph-a-single-powershell-api-for-microsofts-cloud-services/)

Microsoft Graph: A single (PowerShell) API

Permission	No access	Limited access	Full access
Block sender (*PermissionToBlockSender*)		✓	✓
Delete (*PermissionToDelete*)		✓	✓
Preview (*PermissionToPreview*)		✓	✓
Allow recipients to release a message from quarantine (*PermissionToRelease*)*			✓
Allow recipients to request a message to be released from quarantine (*PermissionToRequestRelease*)		✓	

(https://4sysops.com/archive quarantine-policies-in-microsoft-365/)

Configure Quarantine Policies in Microsoft 365 (https://4sysops.com/archi

(https://4sysops.com/archives/ventoy-boot-multiple-iso-images-from-a-single-usb-stick/)
(https://4sysops.com/archives/how-to-identify-virtual-nics-in-hyper-v-virtual-machines/)

0 COMMENTS

Leave a reply

Your email address will not be published. Required fields are marked *

Comment

Name *

Email *

Website

☐ Notify me of followup comments via e-mail. You can also subscribe (https://4sysops.com/comment-subscriptions/?srp=1557699&srk=&sra=s&srsrc=f) without commenting.

POST COMMENT

☐ Receive new post notifications

Subscribe to newsletter

Email Address

Subscribe

Follow 4sysops

(http://twitter.com/4sysops/) (http://www.facebook.com/4sysops) (https://www.linkedin.com/company/4sysops/) (https://4sysops.com/feed/)

© 4sysops 2006 - 2022